강의 · 실습 키트 포함

나만의 AI 업무 시스템

노트북 한 대로 만드는 SSR+ Claude 대시보드

가상 회사 '모아랩' 실습 키트로 스킬부터 대시보드까지 직접 세운다

AI 성능이 아니라 일하는 환경이 생산성을 정한다.
사람은 프롬프트를 붙여 넣고 확인하며, 나머지는 Claude Code가 한다.

대시보드는 결과이고 바닥은 네 단계 세팅이다

이 강의 전체를 관통하는 세 가지 주장

1

화면보다 바닥을 먼저 깐다

스킬, 세컨드 브레인, 루틴, 익스텐션을 깔아야 AI가 맥락을 갖고 움직인다.

2

노트북 한 대와 작업 폴더 하나면 충분하다

상시 PC와 Hermes 없이 로컬 예약 작업으로 돌리고, 꼭 필요한 일만 클라우드로 보낸다.

3

도구를 배우지 않고 만들라고 시킨다

설치, 탐색기, 커넥터 찾기, 대시보드까지 프롬프트로 Claude Code에 맡긴다.

목표: 실습 키트로 영상 속 시스템을 오늘 내 노트북에서 재현한다

일이 줄지 않는 이유는 작업 환경의 부재다

SSR+의 네 용어를 먼저 정의한다 — S 스킬, S 세컨드 브레인, R 루틴, + 익스텐션

스킬

업무 단축키

반복 업무의 절차와 참고 자료를 묶은 폴더다. 맥락이 맞으면 저절로 발동한다.

예) 견적서 만들기

세컨드 브레인

AI가 읽는 지식 지도

AGENTS.md와 길잡이 파일로 AI가 자료를 스스로 찾게 한다.

예) 촬영 원본 찾기

루틴

예약된 단축키

정해진 시간에 스킬이나 프롬프트를 자동으로 실행한다.

예) 매일 아침 브리핑

익스텐션

외부 도구 연결

메일, 캘린더, 브라우저, 공공데이터를 AI가 다루게 연결한다.

예) Gmail 커넥터

출발점: 같은 작업을 두 번 이상 시켰다면 스킬로 만든다

대시보드 한 화면에 지식과 운영 현황이 모인다

영상 속 대시보드 구성 — 칸마다 아래 꼬리표가 그 칸을 받치는 단계다

채널과 목표

유튜브와 뉴스레터 현황
이번 달 목표와 캘린더

익스텐션 + 루틴

세컨드 브레인

지식 연결을 보고 검색한다



길잡이 파일이 받친다

메일함

답장이 필요한
메일을 요약한다

익스텐션

스킬

누르면 실행되는 단축키
모델과 프롬프트 선택

스킬

루틴

정해진 시간에 자동 실행
마지막 실행 시각 표시

루틴

네 단계가 쌓인 결과를 대시보드가 보여 준다

작업 폴더(볼트) 하나 위에 네 단계가 올라가고 대시보드가 그것을 모아 보여 준다



단계마다 초급에서 고급으로 오르는 길이 있다

노트북 한 대 기준으로 바꿨다 — 루틴의 중급은 상시 PC 대신 클라우드 예약 작업이다

단계	초급	중급	고급
스킬	스킬 크리에이터에게 말해서 만든다	스킬 폴더에 참고 자료를 넣는다	스킬을 고치는 스킬을 둔다
세컨드 브레인	AGENTS.md에 지침을 적는다	주제별 길잡이 파일을 만든다	나만의 탐색기를 만든다
루틴	로컬 예약 작업을 쓴다	클라우드 예약 작업을 더한다	두 번째 PC가 생기면 Hermes를 둔다
익스텐션	기본 커넥터를 클릭해 연결한다	MCP, CLI, API 순으로 찾는다	필요한 도구를 직접 만든다

영상 화자는 집의 남는 PC에 Hermes를 두었지만, 노트북 한 대라면 그 단계가 필요 없다

00

LAB 0 · 준비

준비 — 실습 키트와 노트북 한 대

사람이 할 일 다섯 가지와 Claude Code가 할 일을 나눈다

실습 키트 하나로 처음부터 끝까지 따라간다

ai-os 볼트 = 옵시디언이 노트를 담는 폴더(vault)이자 Code 탭에서 여는 작업 폴더 — 가상 회사 모아랩의 것



두 예약 작업은 실행 위치와 파일 접근에서 갈린다

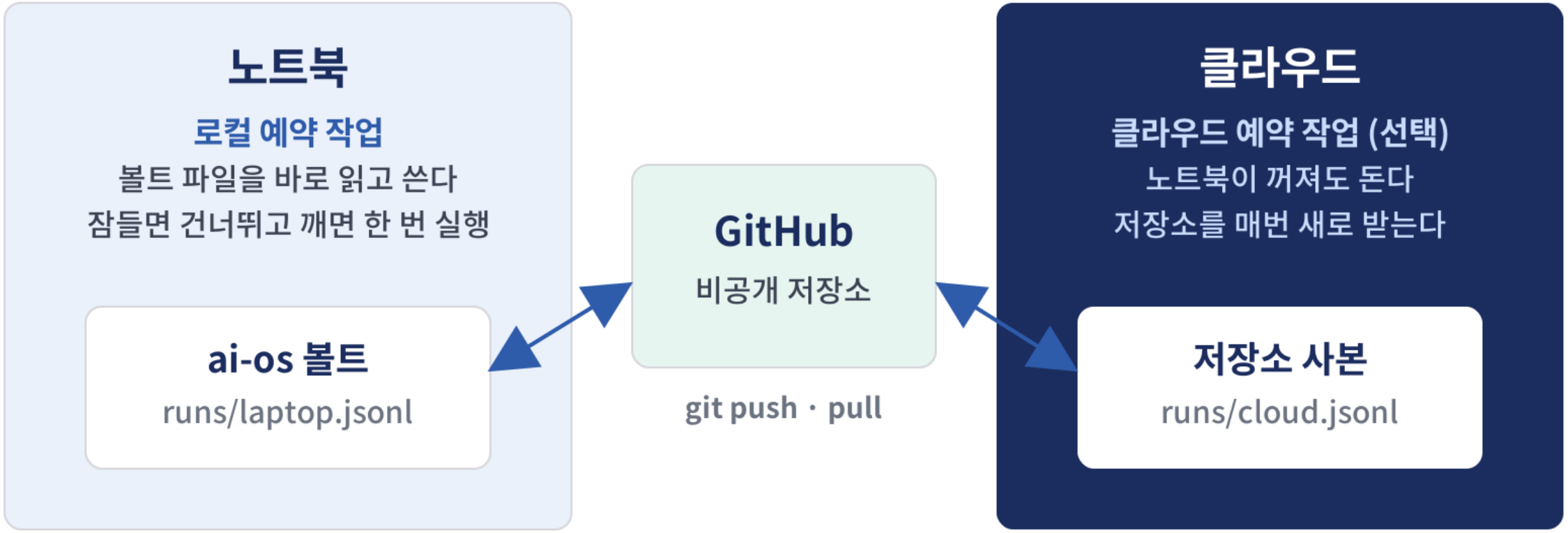
로컬은 데스크톱 앱에서, 클라우드는 claude.ai/code/routines나 /schedule에서 만든다

	로컬 예약 작업	클라우드 예약 작업 (선택)
실행 위치	내 노트북 (데스크톱 앱)	Anthropic 클라우드
노트북이 잠들면	건너뛰고, 깨면 가장 최근 한 번만 실행	상관없이 돈다
볼트 파일	바로 읽고 쓴다	못 읽는다 — GitHub 저장소를 매번 새로 받는다
최소 간격	폼에서는 1시간	1시간
플랜	데스크톱 앱이면 된다	Pro 이상 (연구 미리보기)

이 강의는 로컬 예약 작업을 기본으로 하고, 클라우드는 Lab 6에서 선택으로 다룬다

노트북 한 대라면 Hermes와 Syncthing은 필요 없다

맞출 두 번째 컴퓨터가 없다 — 로컬 예약 작업이 기본이고 클라우드 예약 작업은 선택이다



Hermes와 Syncthing 없이 노트북 한 대로 끝난다

사람이 손으로 할 일은 다섯 가지뿐이다

설치와 작성, 설정은 모두 Claude Code에게 시킨다 — 수강생은 붙여 넣고 확인만 한다

#	사람이 직접 할 일	언제
1	Claude Pro 이상 구독과 로그인	시작 전
2	Claude 데스크톱 앱 설치	시작 전
3	실습 키트 zip 내려받기	시작 전
4	Gmail·캘린더 커넥터 허용 클릭	Lab 4 (선택)
5	GitHub 로그인 승인 클릭	Lab 6 (선택)

Claude Code가 하는 일

- Python, Git, Node 확인과 설치
- 키트 압축 풀기와 저장소 만들기
- 스킬, 길잡이, 대시보드 작성
- 예약 작업 만들기와 오류 해결

환경 준비도 프롬프트 한 장으로 끝낸다

데스크톱 앱 Code 탭에서 새 세션을 열고 폴더로 ai-os를 고른다

[입력] 프롬프트 · Code 탭 · 폴더 ai-os

```
이 실습 환경을 준비해라.
1) python3, git, node 버전을 확인하고 없으면
   설치해라 (맥 Homebrew, 윈도우 winget).
2) python3 _mock/generate_mock.py 로
   오늘 기준 가상 데이터를 다시 만들어라.
3) git 저장소로 만들고 .gitignore에
   .env, node_modules/ 를 넣어라.
4) 첫 커밋을 하고 폴더 트리를 보여 줘라.
```

[출력] 결과

```
기준일 <오늘> — youtube_daily.csv, newsletter_daily.csv,
mail_inbox.json(25건), calendar_events.json(18건) 생성
```

[피드백] 이렇게 확인한다

- 유튜브 데이터 마지막 줄이 오늘 날짜인지 묻는다
- 설치 전에 무엇을 설치할지 알려 주는지 본다
- zip 풀기도 Claude Code에게 시킨다

01

LAB 1 · 스킬

스킬 — 견적서를 단축키로

모아랩 주간 리뷰: 견적서 작성이 매번 30분씩 걸린다

첫 스킬은 스킬 크리에이터에게 말로 주문한다

상황 — 가람물류 2차 미팅 전에 견적서를 보내야 한다

[입력] 프롬프트 · Code 탭 · 폴더 ai-os

```
skill-creator로 견적서 스킬 quote-md를
.claude/skills/에 만들어라.
회의록이나 메일을 주면 02_OPERATIONS/의
사업자 정보와 단가표로 견적서를 md로 만들어
02_OPERATIONS/quotes/에 저장한다.
단가표 아래 규칙을 지키고, 회의록에 없는 값은
추측하지 말고 나에게 묻게 해라.
```

[입력] 시험 · 새 세션에서 이름을 부르지 않고

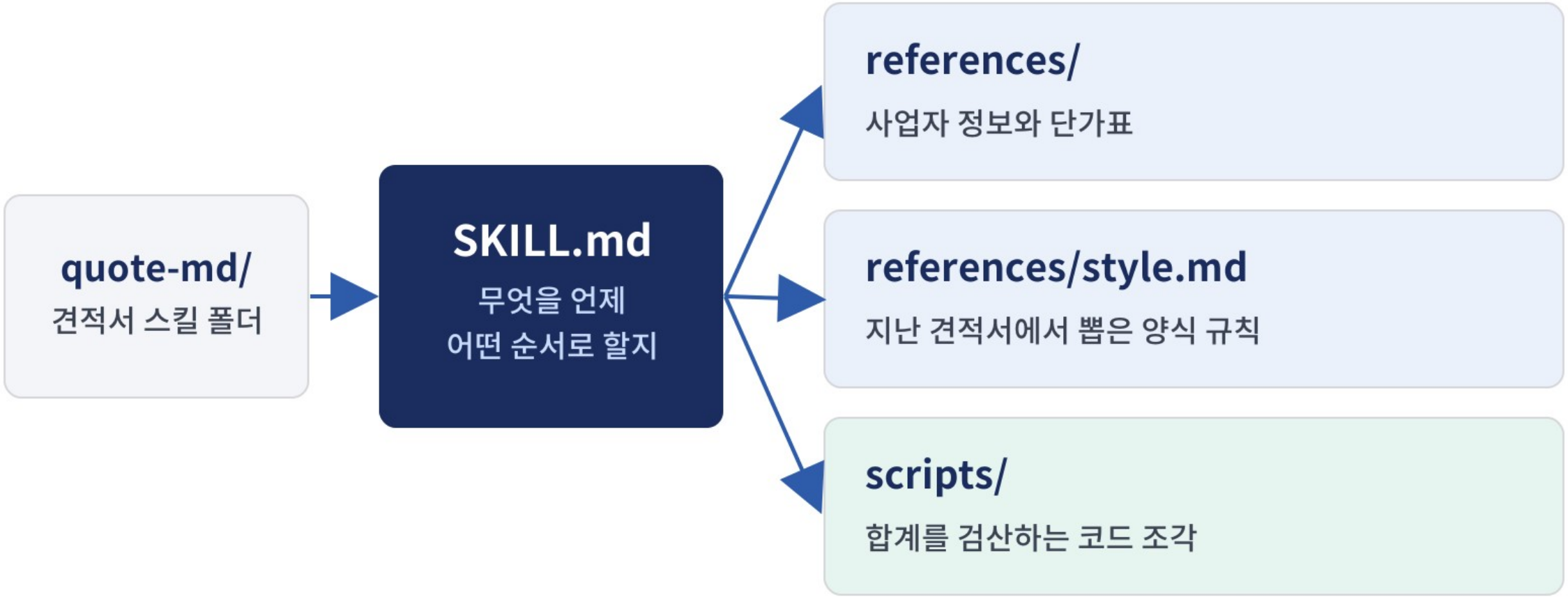
가람물류 킥오프 회의록 보고 견적서 만들어 줘

정답 확인

- 강의 4시간 1,600,000
- 커리큘럼 설계 800,000
- 실습 자료 2회차 600,000
- 합계 3,000,000 (부가세 별도)
- 출장비와 Q&A는 넣을지 묻는다

참고 자료를 넣은 스킬은 결과가 일정하다

SKILL.md는 지시만 하고 세부 규칙과 양식은 하위 폴더에 둔다 — 코드 조각을 넣으면 더 빠르다



스킬은 참고 자료로 키우고 고치는 장치로 지킨다

회사 양식이 바뀌면 예전 스킬은 틀린 결과를 낸다

[입력] 프롬프트 · 중급 · 참고 자료 넣기

```
quote-md를 중급으로 키워라.
references/에 지난 견적서 두 건의
양식 규칙을 style.md로 두고,
SKILL.md 첫 줄에 'references/를
먼저 읽어라'를 넣어라.
합계 검산 한 줄도 남기게 해라.
```

[입력] 프롬프트 · 고급 · 업데이트 절차

```
quote-md 끝에 '업데이트 절차'를
추가해라. 내가 결과를 고치면 차이를
references/changelog.md에 적고,
같은 수정이 두 번 나오면
규칙 자체를 고치자고 제안한다.
```

[피드백] 이렇게 확인한다

- '출장비는 묻지 말고 빼 줘'라고 두 번 고쳐 본다. 두 번째에 규칙 수정 제안이 나오면 성공이다

02

LAB 2 · 세컨드 브레인

세컨드 브레인 — AI가 자료를 찾게

폴더 정리 대신 길잡이 파일을 정리한다

AGENTS.md는 AI가 먼저 읽는 업무 지침서다

CLAUDE.md나 AGENTS.md라는 이름의 파일은 세션을 열 때마다 읽힌다

[입력] 프롬프트 · Code 탭 · 폴더 ai-os

```
README와 폴더 전체를 읽고 모아랩 대표 입장의
AGENTS.md 초안을 써라. 항목은 나는 누구인가,
항상 지킬 규칙, 업무 영역별 길잡이, 하지 말 것.
'_mock/은 직접 고치지 않는다',
'media/raw/ 원본은 옮기지 않는다'를 넣어라.
확실하지 않은 것은 질문 3개로 물어봐라.
CLAUDE.md에는 '@AGENTS.md' 한 줄만 둬라.
```

[출력] 결과 · AGENTS.md (일부)

```
## 업무 영역별 길잡이 — 먼저 여기를 본다
- 고객·견적·회의: _index/business.md
```

CLAUDE.md와의 관계

- Claude Code는 CLAUDE.md를 자동으로 읽는다
- '@AGENTS.md' 한 줄로 AGENTS.md를 함께 불러온다
- 다른 에이전트 도구도 AGENTS.md를 읽는다

길잡이 파일은 링크로 자료의 위치를 알린다

AGENTS.md가 대문, _index/ 아래 파일이 주제별 안내 페이지 역할을 한다



길잡이가 있으면 모호한 질문도 바로 닿는다

키트의 inbox/와 media/raw/에는 이름이 엉망인 파일이 일부러 섞여 있다

[입력] 프롬프트 · Code 탭 · 폴더 ai-os

```
_index/에 content, business,
research, ops 길잡이를 만들어라.
inbox/ 메모는 읽고 관련 프로젝트에
연결하되 파일은 옮기지 않는다.
어디에도 안 붙는 파일은
_index/orphans.md에 이유와 적어라.
```

[입력] 시험 · 새 세션에서

ep42 A캠 최종 원본 어디 있어?

[출력] 맞는 답

```
media/raw/20261024_ep42_A-cam_*.mp4
근거: inbox/촬영메모.txt 'A캠이 진짜 최종'
```

에피소드 · 촬영 원본 찾기

- 파일 탐색기 검색은 엉뚱한 파일부터 보여 주지만, 길잡이를 읽은 AI는 원본과 메모를 함께 찾는다

탐색기는 내 지식을 내 방식으로 보여 준다

옵시디언을 이미 쓴다면 건너뛰어도 된다 — 그래프 뷰와 검색은 옵시디언이 한다

[입력] 프롬프트 · Code 탭 · 폴더 ai-os

```
dashboard/explorer/에 로컬 웹앱을 만들어라.
AGENTS.md를 중심으로 길잡이와 위키링크를
그래프로 그리고 계층 뷰로도 바꾸게 해라.
파일을 누르면 미리 보기를 띄우고
검색창은 파일명과 본문을 함께 찾는다.
실행까지 하고 브라우저 주소를 알려 줘라.
```

[피드백]

- 검색창에 'A캠'을 넣어 원본 목록이 바로 나오는지 본다



03

LAB 3 · 루틴

루틴 — 노트북이 깨어 있을 때 저절로

로컬 예약 작업으로 충분하다. 놓친 실행은 깨어날 때 한 번 몰아서 돈다

모닝 브리핑은 데스크톱 세션에서 말로 예약한다

Code 탭의 예약(Scheduled) 메뉴에서 만들어도 되고, 세션에서 요청해도 된다

[입력] 프롬프트 · Code 탭 · 폴더 ai-os

```
매일 오전 8시 50분 로컬 예약 작업 morning-briefing을
만들어라.
1) _mock/의 메일과 일정, 01_PROJECTS/, 목표를 읽는다
2) briefings/YYYY-MM-DD.md에 오늘 일정, 답장이 필요한
   메일(스팸·알림 제외), 마감 임박, 목표 진행률을 쓴다
3) 낮 12시 이후에 돌면 제목에 '[놓친 브리핑]'을 붙인다
4) 끝나면 runs/laptop.jsonl에 한 줄을 남긴다
만든 뒤 바로 한 번 실행해라.
```

[출력] 결과 · runs/laptop.jsonl

```
{"routine":"morning-briefing","device":"laptop","result":"완료"}
```

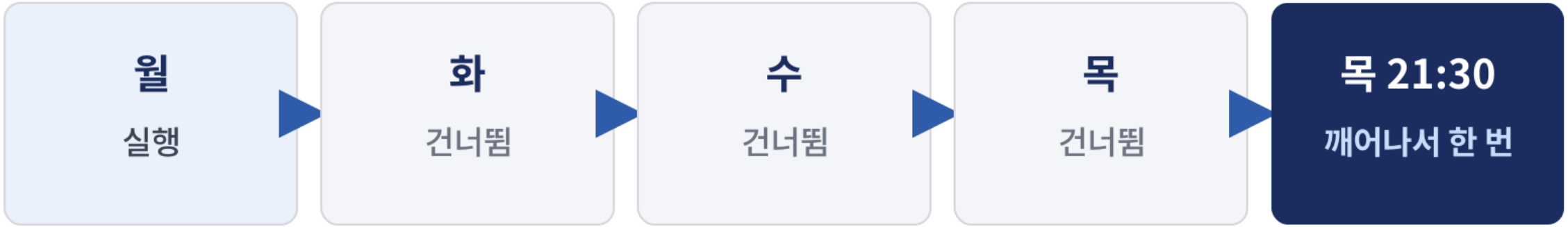
처음 실행할 때

- 권한을 물으면 항상 허용을 고른다
- 그래야 다음 실행이 멈추지 않는다
- 답장 필요 메일이 9건인지 본다

놓친 실행은 깨어날 때 한 번만 몰아서 돈다

로컬 예약 작업은 노트북이 잠든 시각을 건너뛴다 — 그래서 프롬프트에 안전장치를 넣는다

매일 08:50 모닝 브리핑 · 노트북을 3일 덮어 둔 경우



놓친 것 중 가장 최근 한 번만 실행한다 (7일 이내)

그래서 프롬프트에 안전장치를 넣는다: 낮 12시 이후에 돌면 제목에 '[놓친 브리핑]'

유휴 잠자기는 Keep computer awake로 막을 수 있지만, 덮개를 닫으면 잠든다

주간 정리 루틴이 길잡이를 최신으로 지킨다

새로 생긴 파일을 길잡이에 연결하지 않으면 세컨드 브레인은 금방 낡는다

[입력] 프롬프트 · Code 탭 · 폴더 ai-os

```
매주 금요일 오후 5시 로컬 예약 작업
weekly-tidy를 만들어라. 이번 주 새 파일을
찾아 _index/ 길잡이에 연결하고, 연결할 곳이
없으면 orphans.md에 추가한다.
파일은 옮기거나 지우지 않는다.
끝나면 runs/laptop.jsonl에 한 줄을 남긴다.
```

[피드백]

- inbox/에 메모 하나를 만들고 Run now로 돌려 길잡이에 붙는지 본다

노트북 설정 두 가지

- Keep computer awake를 켜면 유휴 잠자기를 막는다
- 덮개를 닫으면 여전히 잠든다
- 기록의 '건너뜀'은 고장이 아니다

04

LAB 4 · 익스텐션

익스텐션 — 외부 도구를 손발로

실습은 가상 데이터로 하고, 실제 커넥터는 선택으로 붙인다

가상 데이터를 커넥터 자리에 끼워 실습한다

_mock/은 Gmail, 캘린더, YouTube API, 뉴스레터 플랫폼을 흉내 낸 파일이다

[입력] 프롬프트 · 가상 데이터 규칙

```
_mock/README.md를 읽고, 실습 동안
_mock/ 파일을 Gmail, 캘린더,
YouTube API 대신 쓴다는 규칙을
AGENTS.md에 한 줄로 넣어라.
나중에 실제로 바꿀 때 고칠 곳을
_index/ops.md에 적어라.
```

[입력] 프롬프트 · 실제 커넥터 확인 (선택)

```
지금 쓸 수 있는 커넥터를 확인해라.
Gmail과 Calendar가 보이면
오늘 일정과 읽지 않은 메일 5건의
제목만 보여 줘라.
메일 본문은 가져오지 마라.
```

실제 커넥터는 사람이 한 번 허용한다

- claude.ai 설정의 커넥터에서 구글 허용 버튼을 누른다. 안 보이면 Claude Code에게 '/mcp로 상태를 보여 줘'라고 한다

연결 방법은 쉬운 것부터 어려운 순서로 찾는다

커넥터를 클릭하고, 없으면 MCP, CLI, API 순으로 내려가고, 마지막에 직접 만든다



커넥터 찾기 자체를 스킬로 만들어 둔다

국내 서비스는 기본 커넥터에 없는 경우가 많다

[입력] 프롬프트 · Code 탭 · 폴더 ai-os

```
connector-finder 스킬을 만들어라.
내가 'X를 연결하고 싶다'고 하면
MCP 서버 → CLI → API 순서로 찾고
후보마다 인증 방식, 비용, 신뢰도를 표로 낸다.
내가 고르면 설치와 연결 시험까지 하되
키 입력은 내가 직접 한다.
```

[입력] 시험

국세청 사업자등록 상태조회를 연결하고 싶다

확인할 것

- 후보 비교표가 나온다
- 키 발급처럼 사람이 할 단계가 따로 표시된다
- 키를 대화창에 붙여 넣으라고 하지 않는다

맞는 도구가 없으면 직접 만들어 쓴다

만들고 싶은 기능을 말하면 에이전트가 준비물과 절차를 안내한다

[입력] 프롬프트 · Code 탭 · 폴더 ai-os

```
이 볼트와 주간 리뷰를 보고 내가 반복해서
불편해한 지점을 찾아라. 직접 만들면 좋을
익스텐션 후보 5개를 내고, 후보마다
만드는 시간과 준비물을 적어라.
```

모아랩이라면

- 촬영 원본을 날짜순으로 보는 라이브러리
- 협업 제안 메일 분류기

에피소드 · 마크다운 뷰어

- 노션과 옵시디언이 불편해 직접 뷰어를 만들었다
- 한 창에 파일 두세 개를 나란히 본다
- 대시보드의 라이브러리도 같은 방식이다

05

LAB 5 · 대시보드

대시보드 — 전체를 한 화면에

데이터 계약 → 수집 스크립트 → 화면 순서로 만든다

화면보다 데이터 계약을 먼저 정한다

화면부터 만들면 숫자가 가짜로 채워진다

[입력] 프롬프트 · Code 탭 · 폴더 ai-os

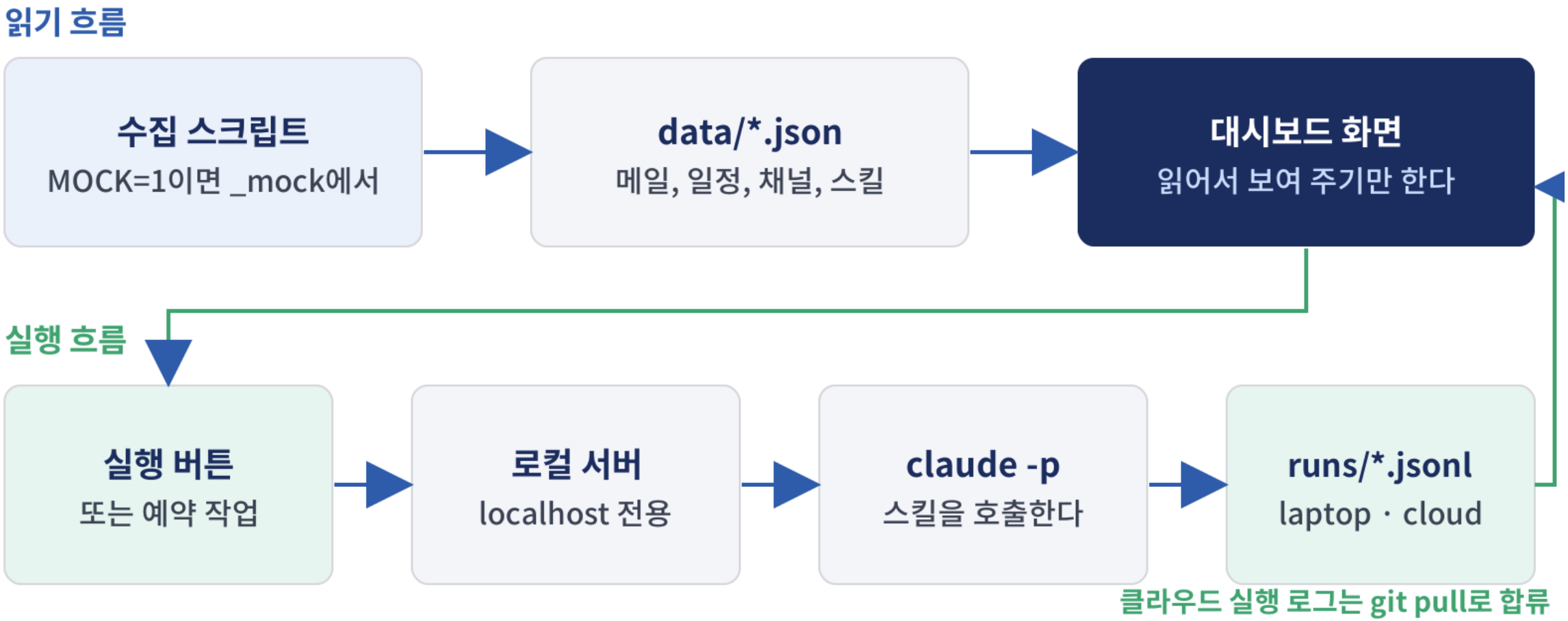
```
dashboard/data/에 JSON 스키마와 예시를 만들어라.
- channels.json: 현재값, 비교값과 비교 날짜,
  수집 시각, 갱신 기한(24시간)
- goals.json: 기간, D-day, 목표와 현재값
- calendar.json: 이번 달 일정과 주차별 일정 수
- mail.json: 최근 20건과 확인 시각
- skills.json: .claude/skills/ 스캔 결과
- routines.json: 실행 위치(laptop/cloud), 일정
- brain.json: _index 영역별 파일 수와 목록
실행 결과는 runs/<위치>.jsonl에 쌓인다.
스키마는 data/SCHEMA.md에 표로 정리해라.
```

[피드백]

- SCHEMA.md에서 빠진 칸을 지금 고친다
- 나중에 고치면 세 곳을 다시 손본다

대시보드는 읽기만 하고 실행은 로컬 서버가 맡는다

실습에서는 수집 스크립트가 _mock/을 읽고, 실행 결과는 runs/laptop·cloud 로그에 쌓인다



수집 스크립트가 계약대로 값을 채운다

MOCK=1이면 가상 데이터를, MOCK=0이면 커넥터와 API를 읽는다

[입력] 프롬프트 · Code 탭 · 폴더 ai-os

```
1단계 JSON을 채우는 수집 스크립트를
dashboard/collect/에 만들어라.
MOCK=1이면 _mock/에서 읽고, MOCK=0 자리도 만들어 둬라.
채널 값은 data/history/에 날마다 스냅샷으로 남긴다.
노트북이 며칠 꺼졌으면 '마지막 스냅샷 대비(N일 전)'로
계산한다. 모닝 브리핑의 첫 단계로 넣어라.
```

[출력] 결과 · data/channels.json (일부)

```
{"youtube":{"now":...,"compare":...,"compare_date":"...",
 "collected_at":"...","stale_after_h":24}}
```

왜 스냅샷인가

- API는 현재값만 준다
- 저장해 두어야 비교할 수 있다
- 노트북이 쉰 날은 N일 전과 비교한다

화면은 마지막에 씌우고 모양은 자유롭게 한다

영상과 똑같을 필요는 없다 — 3열 배치와 포인트 색 하나만 정한다

[입력] 프롬프트 · Code 탭 · 폴더 ai-os

```
dashboard/에 로컬 웹앱을 만들어라. data/의 JSON과 runs/만 읽는다. 3열, 포인트 색 하나.
- 왼쪽: 채널 카드(현재값, 비교값, 수집 시각, 기한을 넘기면 '갱신 필요'), 목표와 D-day, 월간 캘린더, 최근 파일
- 가운데: 세컨드 브레인 영역별 파일 수와 탐색기
- 오른쪽: 메일(안 읽은 수, 최근 20건), 스킬(검색, 상태, 실행 버튼),
  루틴(runs/의 모든 로그를 모아 실행 위치, 일정, 최근 결과)
실행 버튼은 로컬 서버가 claude -p로 스킬을 부르고 결과를 runs/laptop.jsonl에 남긴다.
실행은 localhost에서만 받는다. npm run dashboard 하나로 뜨게 하고 주소를 알려 줘라.
```

[출력] 결과 · 브라우저

```
http://localhost:<포트>
3열 대시보드가 열린다
```

[피드백]

- quote-md 버튼으로 견적서가 생기는지 본다

디자인보다 세 가지 동작을 점검한다

대시보드가 완성됐는지는 모양이 아니라 아래 세 가지로 판단한다

1

숫자가 살아 있는가

모닝 브리핑을 Run now로 돌린 뒤 채널과 메일 값, 수집 시각이 바뀐다.

2

스킬 버튼이 실제로 도는가

quote-md 버튼으로 견적서가 quotes/에 생기고 카드에 완료가 표시된다.

3

다른 곳에서 돈 루틴도 보이는가

Lab 6을 했다면 클라우드에서 돈 주간 리뷰가 루틴 칸에 cloud로 보인다.

세 번째가 안 되면 git pull이 브리핑 앞에 들어 있는지, main에 푸시됐는지 본다

06

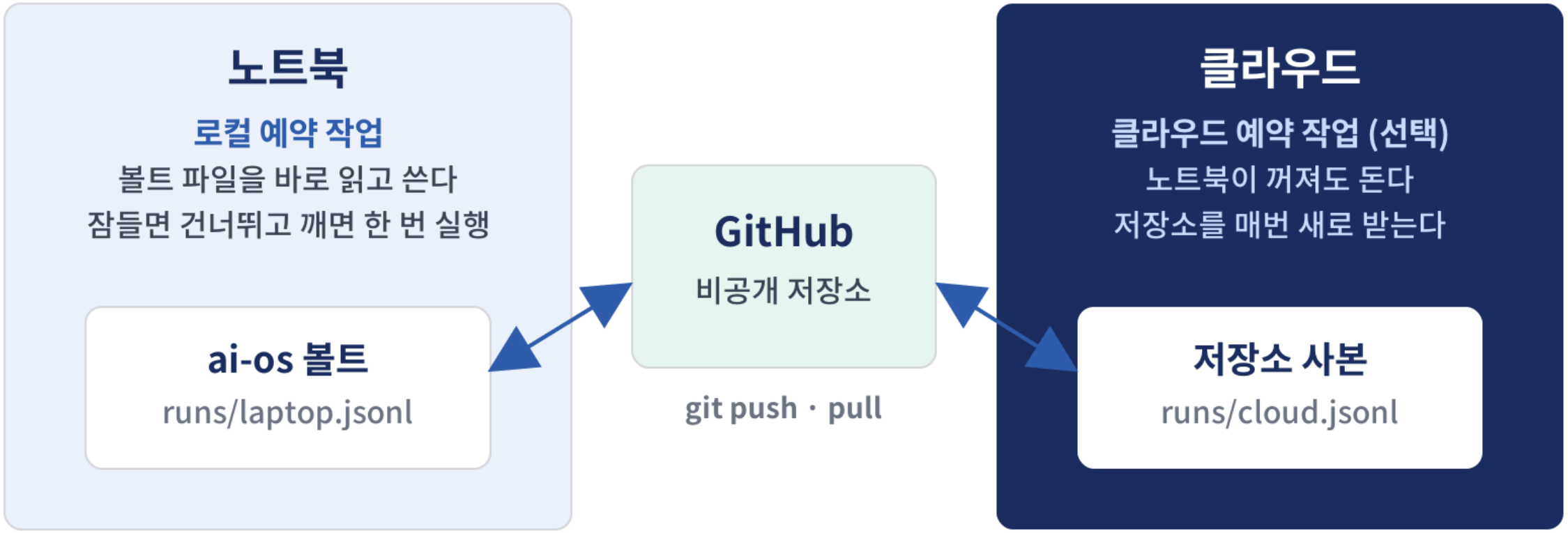
LAB 6 · 선택

클라우드 — 노트북을 닫아도 도는 일

볼트를 비공개 GitHub 저장소에 올리는 것이 전제다

노트북과 클라우드는 GitHub를 사이에 두고 주고받는다

노트북은 push와 pull로, 클라우드 작업은 clone과 push로 같은 저장소를 쓴다



Hermes와 Syncthing 없이 노트북 한 대로 끝난다

클라우드 작업은 저장소를 받아서 일하고 다시 올린다

클라우드 예약 작업은 노트북 파일을 못 읽는다 — Pro 이상, 연구 미리보기 기능이다

[입력] 프롬프트 · 6-1 · 저장소 올리기

```
gh CLI가 없으면 설치하고, 로그인이
필요하면 gh auth login을 실행해라
(브라우저 승인은 내가 한다).
이 볼트를 비공개 저장소
moalab-ai-os로 올리고
.env가 빠졌는지 확인해라.
```

[입력] 프롬프트 · 6-2 · 예약 → 새 작업 → Cloud

```
(저장소 moalab-ai-os, 매주 월 07:00)
이 저장소에서 지난주 briefings/와
runs/를 읽고 주간 리뷰를
briefings/weekly-YYYY-Www.md로 써라.
runs/cloud.jsonl에 한 줄을 남기고
main 브랜치에 커밋해 푸시해라.
```

올리기 전에

- 실제 업무 볼트는 고객 실명과 계약서를 빼고 올린다

왜 main인가

- 기본은 claude/ 브랜치라 노트북의 git pull로 못 받는다

노트북 루틴의 처음과 끝에 git을 붙여 잇는다

노트북은 pull로 클라우드 결과를 받고, push로 자기 결과를 올린다

[입력] 프롬프트 · Code 탭 · 폴더 ai-os

```
morning-briefing 예약 작업의 맨 앞에
'git pull' 단계를 넣고, 맨 끝에 변경 사항을
커밋해 푸시하게 고쳐라.
충돌이 나면 멈추고 브리핑에 그 사실을 적어라.
```

[입력] 시험 · 클라우드 작업을 Run now로 돌린 뒤

```
git pull 하고 runs/cloud.jsonl 마지막 줄 보여 줘
```

Hermes가 필요해지는 때

- 항상 켜 둘 두 번째 컴퓨터가 생겼을 때
- 메신저로 밖에서 일을 시키고 싶을 때
- Claude 외의 모델로 돌리고 싶을 때

4주면 노트북 한 대로 시스템을 재현할 수 있다

주마다 한 층씩 쌓고 완료 기준으로 점검한다

1주 Lab 0 · 1	2주 Lab 2	3주 Lab 3 · 4	4주 Lab 5 · 6
키트를 풀고 견적서 스킬을 만든다	AGENTS.md와 길잡이 파일을 만든다	브리핑 루틴과 커넥터를 연결한다	대시보드와 클라우드 작업을 만든다
완료 기준 합계 300만 원이 맞고 스킬이 저절로 발동한다	완료 기준 모호한 질문에 맞는 파일을 바로 찾는다	완료 기준 놓친 날도 표시되며 쌓인다	완료 기준 화면 점검 세 가지를 통과한다

막히면 증상을 그대로 Claude Code에게 말한다

자주 나오는 다섯 가지 증상과 원인

증상	원인	Claude Code에게 할 말
기록에 '건너뜀'	그 시각에 노트북이 잠들어 있었다	정상이다. 깨어나면 한 번 돈다
예약 작업이 멈춰 있다	권한 확인을 기다린다	그 실행 세션을 열어 항상 허용
커넥터가 안 보인다	연결 안 됨 또는 인증 만료	/mcp로 상태를 보여 줘
클라우드 결과가 없다	claude/ 브랜치에 푸시됐다	원격 브랜치를 main에 병합해 줘
가상 데이터가 옛날 날짜	생성기를 다시 안 돌렸다	generate_mock.py 실행해 줘

내 업무에 옮기기 전에 네 질문에 답한다

각 질문의 답이 곧 첫 번째 스킬, 길잡이, 루틴, 커넥터 후보가 된다

Q1	지난 한 달 동안 두 번 이상 시킨 업무는 무엇인가?	**스킬**
Q2	AI가 자주 못 찾는 자료는 어디에 흩어져 있는가?	**세컨드 브레인**
Q3	노트북을 닫아 둔 시간에도 꼭 돌아야 하는 일은 무엇인가?	**루틴**
Q4	기본 커넥터에 없지만 꼭 연결할 국내 서비스는 무엇인가?	**익스텐션**

바닥부터 쌓고 화면은 마지막에 씌운다

이 강의의 세 가지 결론

1

바닥부터 쌓는다

스킬, 세컨드 브레인, 루틴, 익스텐션을 차례로 깔고 대시보드는 마지막에 만든다.

2

노트북 한 대로 시작한다

로컬 예약 작업이 기본이고, 노트북을 닫아도 돌아야 하는 일만 클라우드로 보낸다.

3

만들라고 시킨다

설치부터 대시보드까지 프롬프트로 Claude Code에 맡기고 사람은 확인한다.

과제: 실습이 끝나면 ai-os를 복사해 내 업무 폴더로 바꾸고 MOCK=0으로 돌린다